

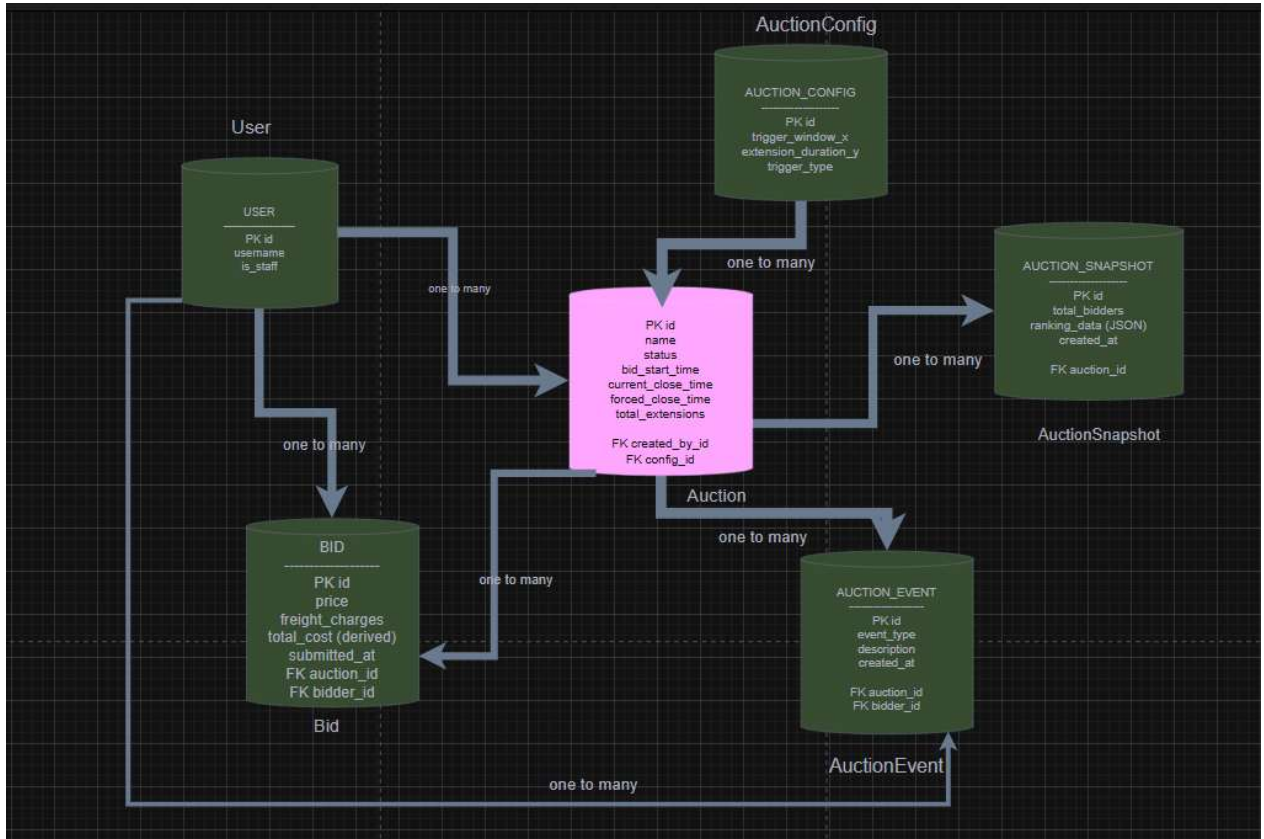
Schema Design for Database Tables

Table of contents

1. Overview
2. Tables
3. Relationships
4. Business Logic
5. Indexing Strategy
 - 5.1 User Table Indexing
 - 5.2 Auction Table Indexing
 - 5.3 Bid Table Indexing
 - 5.4 Auction Event Table Indexing
 - 5.5 Auction Config Table Indexing
 - 5.6 Auction Snapshot Table Indexing
6. Data Flow
7. Scalability
8. Key Strengths

1. Overview

This schema is designed for an auction system where users create auctions and suppliers place bids. The system stores bids, tracks events, and maintains auction status.



2. Tables

◆ USER (auth_user)

Stores system users.

Field	Type	Description
id	PK	User ID
username	VARCHAR	Username
is_staff	BOOLEAN	Admin/Supplier

◆ AUCTION_CONFIG

Stores auction extension rules.

Field	Type	Description
id	PK	Config ID
trigger_window_x	INT	Last X minutes

Field	Type	Description
extension_duration_y	INT	Extend by Y minutes
trigger_type	VARCHAR	Trigger condition

◆ AUCTION

Stores auction details.

Field	Type	Description
id	PK	Auction ID
name	VARCHAR	Auction name
status	VARCHAR	ACTIVE / CLOSED
bid_start_time	TIMESTAMP	Start time
current_close_time	TIMESTAMP	Dynamic close
forced_close_time	TIMESTAMP	Final deadline
created_by_id	FK → User	Creator
config_id	FK → AuctionConfig	Config used

◆ BID

Stores supplier bids.

Field	Type	Description
id	PK	Bid ID
auction_id	FK → Auction	Auction reference
bidder_id	FK → User	Supplier
price	DECIMAL	Bid price
submitted_at	TIMESTAMP	Time

👉 Note: One user can place multiple bids.

◆ AUCTION_EVENT

Stores system events.

Field	Type	Description
id	PK	Event ID
auction_id	FK	Auction
bidder_id	FK	User
event_type	VARCHAR	Event type
created_at	TIMESTAMP	Time

◆ AUCTION_SNAPSHOT

Stores ranking snapshots.

Field	Type	Description
id	PK	Snapshot ID
auction_id	FK	Auction
ranking_data	JSON	Rankings
created_at	TIMESTAMP	Time

3. Relationships

- User → Auction → **1 : Many**
 - User → Bid → **1 : Many**
 - Auction → Bid → **1 : Many**
 - AuctionConfig → Auction → **1 : Many**
 - Auction → AuctionEvent → **1 : Many**
 - Auction → AuctionSnapshot → **1 : Many**
-

4. Business Logic

4.1 Ranking Logic

- ORDER BY total_cost ASC, submitted_at ASC
- Lowest cost = L1
- Tie breaker = earlier submission

4.2 Extension Logic (British Auction)

- If bid placed in last X minutes → extend by Y minutes
- Constraint: current_close_time ≤ forced_close_time

4.3 Bid Revision Logic

- Multiple bids allowed per supplier
 - Latest bid used for ranking
 - Old bids stored for audit
-

5. Indexing Strategy

◆ USER Table

- PRIMARY KEY (id)
- UNIQUE (username)

◆ AUCTION Table

- PRIMARY KEY (id)
- INDEX (status, current_close_time)
- INDEX (status, bid_start_time)
- INDEX (created_by_id)
- INDEX (config_id)

◆ BID Table

- PRIMARY KEY (id)
- INDEX (auction_id, bidder_id, submitted_at)
- INDEX (auction_id, submitted_at)
- INDEX (bidder_id)

Helps in:

- Fast latest bid retrieval
- Ranking queries

◆ AUCTION_EVENT Table

- PRIMARY KEY (id)
- INDEX (auction_id, created_at)
- INDEX (bidder_id)

◆ AUCTION_CONFIG Table

- PRIMARY KEY (id)

◆ AUCTION_SNAPSHOT Table

- PRIMARY KEY (id)
- INDEX (auction_id, created_at)

6. Data Flow

1. Auction created by user
2. Suppliers place bids

3. System evaluates ranking
4. If condition met → extend auction
5. Events logged for every action
6. Snapshots captured periodically
7. Auction closes

7. Scalability

Stage	Action
Small	SQLite
Medium	PostgreSQL
Large	Partitioning + Indexing
Enterprise	Read Replicas

8. Key Strengths

- Supports real-time bidding
- Maintains complete audit trail
- Handles bid revisions
- Implements dynamic extension logic
- Scalable and optimized